__/__/____

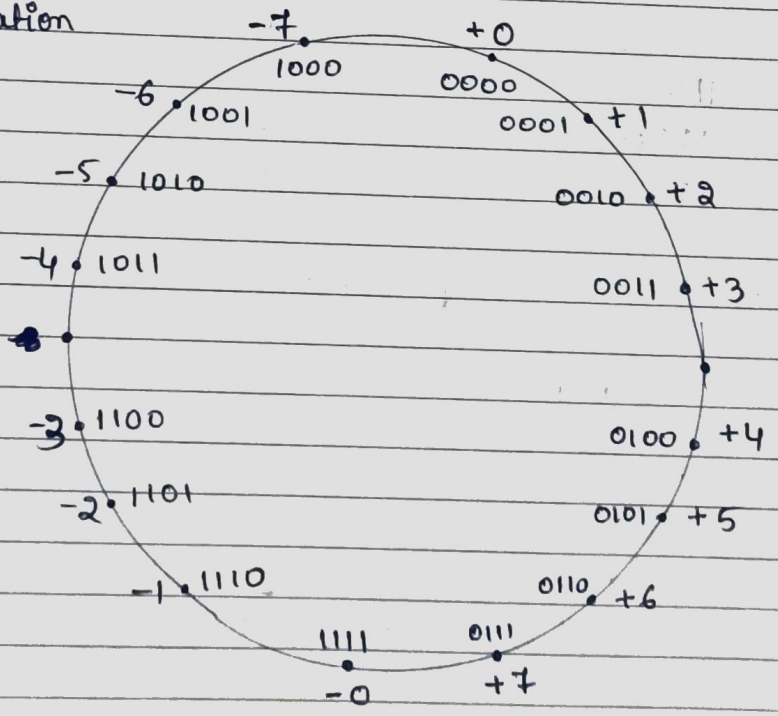
Pipeline and Superscalar Operation.

Signed Integer
- Sign and magnitude
- One's complement
- Two's complement.

Assumptions.
- 4-bit machine world
- 16 different values can be represented
- Roughly half are positive & another half are negative.

Sign and magnitude representation.

1's complement Representation



Binary, Signed-integer Representations

$b_3 b_2 b_1 b_0$	Sign & magnitude	1's complement	2's complement
0000	+0	+0	+0
0001	+1	+1	+1
0010	+2	+2	+2
0011	+3	+3	+3
0100	+4	+4	+4
0101	+5	+5	+5
0110	+6	+6	+6
0111	+7	+7	+7
1000	-0	-7	-0
1001	-1	-6	-7
1010	-2	-5	-6
1011	-3	-4	-5
1100	-4	-3	-4
1101	-5	-2	-3
1110	-6	-1	-2
1111	-7	-0(-8)	-1

* Addition of numbers using 2's complement

- $3 + 4 = ?$
  $= 7$

```
 0100
 0011
------
 0111
```

- $-4 + (-3) = -7$

```
 1100
 1101
------
11001
```

__/__/____

⊛ Subtraction of numbers in 2's complement.

C-1 .

```
  4          0100  → minuend
- 3          0011  → subtrahend.
  1
```

Steps :- 1) Convert subtrahend to 2's complement
2) Add minuend and 2's complement of subtrahend

```
  3 → 0011 → 2's complement → 1101
  0100
  1101
 10001   ⇒ 1
```

C-2 .

```
-4 + 3 = -1
-4          0100  → minuend
+3          0011  → Subtrahend.
-1
```

Steps :- 1) Convert minuend to 2's complement
2) Add them together.

```
 4 → 0100 → 2's complement → 1100
 1100
 0011
 1111   ⇒ -1
```

C-3 .

```
-3 - (-4) = 4
 -3          1101  → minuend
-(-4)        1001  → Subtrahend.
 4
```

```
-4 → 1001 → 2's complement → 0111
 1101
 0111
10100   ⇒ 4.
```

- $+2 - (4) = -2$ 0010 / 0100 → 2's complement 1011 + 1 = 1100

  0010
  1100
  1110 ⇒ -2

- $+6 - (+3)$ 0110 − 0011 → 2's complement → 1101

  0110
  1101
  1 0011 ⇒ $+3$

- $-7 - (-5)$ 1001 − 1011 → 2's complement → 0101

  1001
  0101
  1110 ⇒ -2

- $-7 - (+1)$ 1001 − 0001 → 2's complement → 1111

  1001
  1111
  1 1000 ⇒ -8

- $+2 - (-3)$ 0010 − 1101 → 2's complement → 0011

  0010
  0011
  0101 ⇒ 5

Overflow

Overflow in unsigned numbers.

→ Case 1 :

	-7	1001
	-3	1101
$-2^4\ 2^3\ 2^2\ 2^1\ 2^0$	-10	10110

1	0	1	1	0

$1 \times -16 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1$

$-16 + 4 + 2 = -10$

But ~~the~~ it will store the result in 4-bit register, the stored number will be the wrong answer, which will lead to overflow.

→ Case 2 :

7	0111
1	0001
-8	1000

Overflow case.

→ Case 3 :

7	0111
-7	1001
0	10000

It is not a overflow condition as the result stored in 4-bit register is the actual result.

→ Case 4 :

2	0010
4	0100
6	0110

Not a overflow condition.

Memory locations, Addresses and operation

Big-Endian and Little-Endian Assignments

- **Big-Endian** :- Lower byte addresses are used for the most significant bytes of the word.
- **Little-Endian** :- [opposite ordering] Lower byte addresses are used for the less significant of the word.

Big-Endian Representation

Word add	Byte Add			
0	0	1	2	3
4	4	5	6	7
	⋮	⋮	⋮	⋮
2^k-4	2^k-4	2^k-3	2^k-2	2^k-1

Little-Endian Represent

word add	Byte add.			
0	3	2	1	0
4	7	6	5	4
	⋮	⋮	⋮	⋮
2^k-4	2^k-1	2^k-2	2^k-3	2^k-4

Memory operation

- Load (read or fetch)
  - → Copies from specific memory location to the processor.
  - →
  - → Registers are used
- Store (write)
  - → Transfer information from processor to specific memory location
  - → Overwrite the content in memory.

__/__/__

Register Transfer Notation

- Identify a location by a symbolic name standing for its hardware binary address [Capital letters :- LOC, PLACE, R_1, etc].
- Processor Registers :- R_0, R_1, R_2 ...
- Contents of a location are denoted by square bracket around the name of the location.

Ex:- $R_1 = 10$

$R_2 = 20$

$R_3 = [R_1] + [R_2]$.

Assembly language Notation

- Represent machine instructions and programs.
  - Move LOC, R_1 = $R_1 \leftarrow [LOC]$
  - Add R_1, R_2, R_3 = $R_3 \leftarrow [R_1] + [R_2]$.

Src1 Src2 Destination

CPU Organization

- Single Accumulator
  - Result usually goes to the Accumulator.
  - Accumulator has to be saved to memory quite often.
- General Register
  - Registers hold operands thus reduce memory traffic.
  - Register bookkeeping.
- Stack
  - Operands & results are always in stack

④ Instruction Formats

There are 4 types of Instruction format

- Three address Instruction [Add R_1, R_2, R_3 $[R_3 \leftarrow [R_1]+[R_2]]$]
- Two address Instruction [Add R_1, R_2 $[R_2 \leftarrow [R_1]+[R_2]]$] [move R_1, R_2]
- One address Instruction [Add B $[AC \leftarrow [AC]+[B]]$]
- Zero address Instruction [Used in stack operation]

- Evaluate $(A+B)*(C+D)$

→ Three instruction

Add A, B, R_1	$[R_1 \leftarrow M[A]+M[B]]$
Add C, D, R_2	$[R_2 \leftarrow M[C]+M[D]]$
Mul R_1, R_2, X	$M[x] \leftarrow R_1 * R_2$

→ Two instruction

Mov A, R_1	$[\because R_1 \leftarrow M[A]]$
Add B, R_1	$[\because R_1 \leftarrow R_1 + [B]]$
Move C, R_2	$[\because R_2 \leftarrow M[C]]$
Add D, R_2	$[R_2 \leftarrow R_2 + M[D]]$
[illegible]	
Mul R_1, R_2	$[R_2 \leftarrow R_1 * R_2]$
Mov R_2, X	$[X \leftarrow R_2]$

→ One-address Instruction

Load A	$[AC \leftarrow A]$
Add B	$[AC \leftarrow AC + M[B]]$
Store T	$[M[T] \leftarrow AC]$
Load C	$[AC \leftarrow [C]]$
Add D	$[AC \leftarrow AC + M[D]]$
[illegible] MUL T	$[AC \leftarrow M[T] * AC]$
Store x	$[M[x] \leftarrow AC]$

→ Zero - Address instruction

Push A	[TOS ← M[A]]
Push B	[TOS ← M[B]]
Add	[TOS + (TOS - 1)]
	B + A
	⇓
	[TOS]
Push C	[TOS ← [C]]
Push D	[TOS ← [D]]
Add	[(TOS - 1) + TOS]
	C + D
	⇓
	[TOS]
Mul	[(TOS - 1) * TOS]
	(A+B) * (C+D)
	⇓
	TOS
Pop X	[[X] ← TOS].

* Registers
- Registers are faster
- Shorter instruction.

* Instruction Execution and Straight line sequencing

Address	← 32bit →				
i	0	1	2	3	A
i+4	4	5	6	7	B
⋮	⋮	⋮	⋮		
$i+2^{n-4}$	2^{n-4}	2^{n-3}	2^{n-2}	2^{n-1}	

Add

Sum ← A+B

Straight line Sequencing.

① Branching

Move N, R1	i
Clear R0	i+4
Decrement R1	i+8
Branch>0 LOOP	i+12 [Each time loops takes the same memory size to execute the instruct
Add R1, R0	

② Condition codes

- Condition code flags
- Condition code registers / status reg
- Four commonly used to set to 1 or 0
  - N [Negative]
  - Z [zero]
  - V [overflow]
  - C [Carry].
- Different instructions affect different flag.

A - B

A = 11110000

B = 00010100

```
 1 1 1 1 0 0 0 0
 0 0 0 1 0 1 0 0   —   2's complement
                       1 1 1 0 1 0 1 1
                                     +1
                       ---------------
                       1 1 1 0 1 1 0 0
 ---------------------------------------
 1 1 1 1 0 0 0 0
 1 1 1 0 1 1 0 0
---------------------
1 1 1 0 1 1 0 0 0
```

Carry = 1

Sign = 1

V = 0

Z = 0

Addressing Modes

- The way in which location of the operand is specified in an instruction.

Name	Assembler-syntax	Addressing function
Immediate	#value	operand = Value
Register	Ri	EA = Ri
Absolute [Direct]	LOC	EA = LOC
Indirect	(Ri)	EA = [Ri]
	(LOC)	EA = [LOC]
Index	X (Ri)	EA = [Ri] + X
Base with Index	(Ri, Rj)	EA = [Ri] + [Rj]
Base with Index and offset	X (Ri, Rj)	EA = [Ri] + [Rj] + X
Relative	X (PC)	EA = [PC] + X
Autoincrement	(Ri) +	EA = [Ri] : increment Ri
Autodecrement	- (Ri)	Decrement Ri : EA = Ri

EA = Effective address Value = a signed value

Addressing modes

1) Immediate mode

The use of a constant in

Move #200, R1 R1 ← 200

Move B, R1

Add #6, R1

Move R1, A

- Variable
- Accessing variable

2) Register mode : By name of register

Ex: Move R1 ← R2

3) Absolute Mode : By address of memory location. Ex :- Move LOC, R1.

4) Indirection and Pointers mode : Provide info from where the memory address of the operand can be determined [Effective address].
- Register Indirect : Indicates the register that holds the address of the register that holds the operand.
  MOV (R2), R1
- Register / memory location that contain the address - pointer. Ex : Treasure hunt.

5) Relative Addressing mode :
EA = PC + Relative Address.

6) Direct Addressing mode.
Use given address to access the memory location. Ex :- Move R1, R2

7) Index mode : the EA of the operand is generated by adding a constant value to the contents of a register.
- Index register - general purpose register.
X(R) : EA = X + [R]
- The constant X may be given either as an explicit number or as a symbolic name representing a numeric value.
- X is called displacement.

Indexing and Arrays

- Offset is given as a constant
  ADD 20(R1), R2
- Offset is in the index register
  ADD 1000(R1), R2

7) Indexing and Arrays

- In general, the index mode facilitates access to an operand whose location is defined relative to a reference point within the data structure in which the operand appears.
- Several variations.
  1) (Ri, Rj) : EA = [Ri] + [Rj]
  2) X(Ri, Rj) : EA = [Ri] + [Rj] + X

5) Relative Mode :- The EA is determined by the index mode using the program counter in place of the general purpose register.

- X(PC) = note that X is a signed number
  Ex : Branch > 8 LOOP
- This location is computed by specifying it is an offset from the current value of PC.
- Branch target may be either before or after the branch instruction.

6) Autoincrement mode :- The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the content of this register are automatically incremented to point to the next item in a list.

- (R)+ : The increment is 1 for byte-sized operands, 2 for 16 bit operand and 4 for 32-bit operands

9) Autodecrement : -(R) decrement first, then point to the address.

⊛ Encoding Machine Instruction.

- Instruction must be encoded - binary pattern - called as machine instruction.
- An assumption was made that all instruction are one word length. (instruction)
- OP code : the type of operation to be performed and the type of operands used may be specified using an encoded binary pattern [Only done when both operands are in Registes]

Ex :- Add R_1, R_2 [one word instruction encoding]

OP code	Source	Destination	Other
bits :- 8	7	7	10

Ex :- Add R_1, LOC.

- 8-bit for OP code and registes, 12-bit for LOC - is insufficient.
- Solution :- 2 word instruction length / format encoding

bits - 8 7 7 10

OP code	Source	Destination	Other info
Memory Address [immediate operand] LOC			

bits - 32

- Ex : Add A, B
- If both operands are in memory location, Solution : 3 word instruction encoding.
- This approach results in instruction of variable. If it takes word length of more than one, the it is called CISC [Complex Instruction Set Computer].

If it completes the execution of instruction in one word instruction format, it is called RISC [Reduced Instruction Set Computer].

Unit - 4

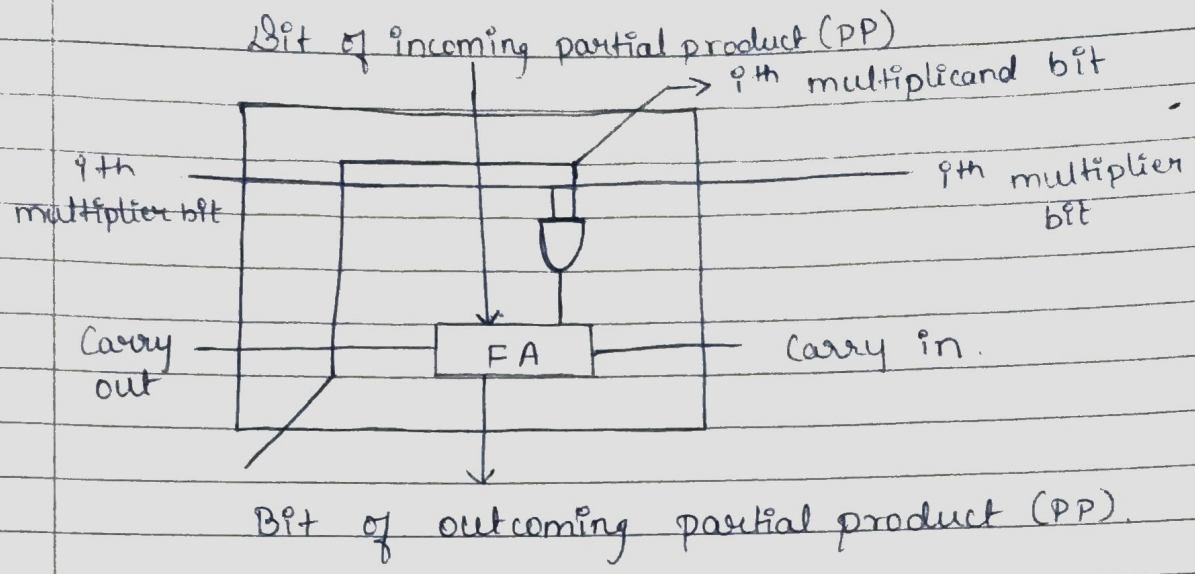
Arithmetic

- Binary multiplication.
  - 4 * 2

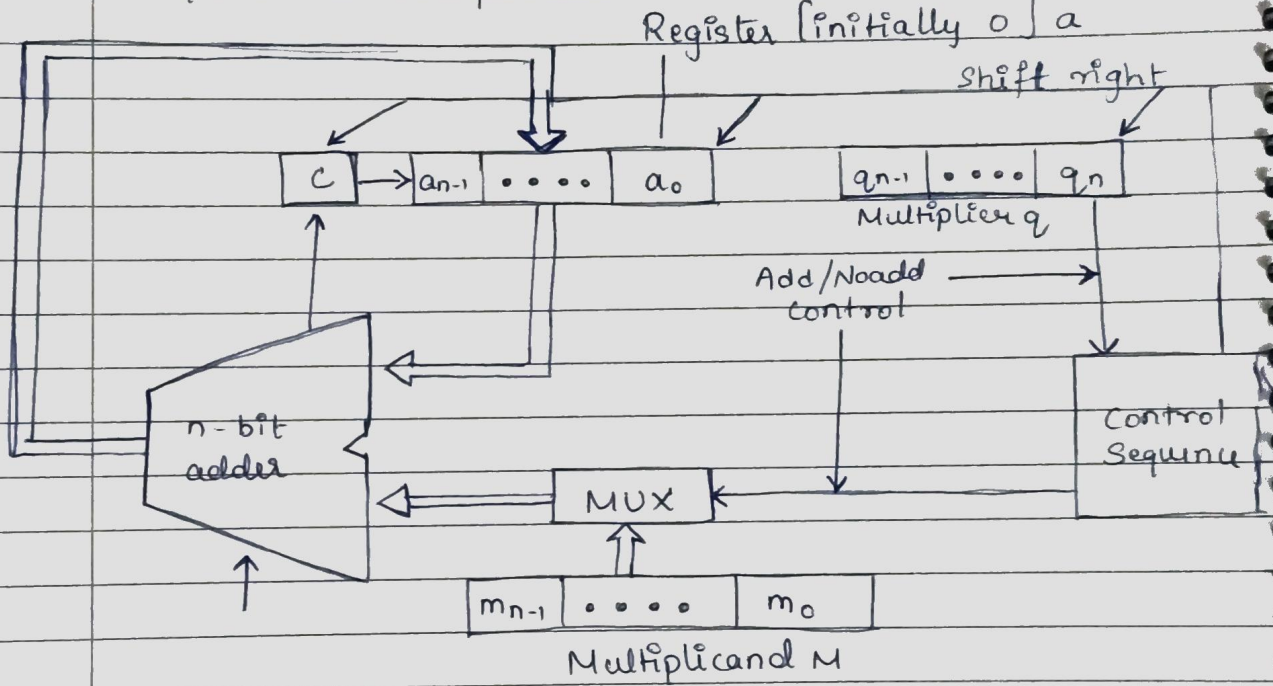
```
  0100   = 4   Multiplicand
  0010   = 2   Multiplier
  ----
  0000
 0100 +
0000 ++
----------
00001000  = 8
```

  - 13 * 11

```
    1101
    1011
    ----
    1101
   1101 +        143
  0000 ++
 1101 +++
----------
10001111  = 143
----------
128 64 32 16 8 4 2 1
```



⊙ Sequential Multiplication.



__/__/____

13 * 11

	Carry	Multiplicand(M)	Multiplier	
	0	1 1 0 1	1 0 1 1	
		Register		
	0	0 0 0 0	1 0 1 1	→ ①
Add	0	1 1 0 1	1 0 1 1	
Right shift	0	0 1 1 0	1 1 0 1	→ ①
Add	1	0 0 1 1	1 1 0 1	
R S	0	1 0 0 1	1 1 1 0	→ ⓪
No Add	0	1 0 0 1	1 1 1 0	
R S	0	0 1 0 0	1 1 1 1	→ ①
Add	1	0 0 0 1	1 1 1 1	
R S	0	1 0 0 0	1 1 1 1	
		1 0 0 0 1 1 1 1	= 143	

14 * 15

		M		
	C	1 1 1 0 (14)	1 1 1 1 (15)	
	0	R 0 0 0 0	1 1 1 1	-①
Add	0	1 1 1 0	1 1 1 1	
R.S	0	0 1 1 1	0 1 1 1	-①
Add	1	0 1 0 1	0 1 1 1	
R.S	0	1 0 1 0	1 0 1 1	-①
Add	1	1 0 0 0	1 0 1 1	
R.S	0	1 1 0 0	0 1 0 1	-①
Add	1	1 0 1 0	0 1 0 1	
R.S	0	1 1 0 1	0 0 1 0	
		1 1 0 1 0 0 1 0 = 210		

Multiplication of Signed Numbers

- -13 × 11

```
      1 0 0 1 1
      0 1 0 1 1
    ---------
      1 0 0 1 1
    1 0 0 1 1
  0 0 0 0 0 0 0
1 0 0 1 1
---------------
1 0 1 1 0 1 1 1 0 0 0 1
```

512 256 64 32 16 1

$= -(143)$

n bit → 2n bits

```
1 1 0 1 1 1 0 0 0 1
                  +
---------------------
0 0 1 0 0 0 1 1 1 0
                  1
---------------------
0 0 1 0 0 0 1 1 1 1
```

128 8 4 2 1

- -11 × 10

0 1 0 1 1 → 11 → 2's comp → -11

```
1 0 1 0 0
        +
    1
-------------
1 0 1 0 1
```

```
      1 0 1 0 1
      0 1 0 1 0
    ---------
0 0 0 0 0 0 0 0 0 0
  1 1 1 1 1 0 1 0 1 +
0 0 0 0 0 0 0 0 0 + +
  1 1 1 0 1 0 1 + + +
0 0 0 0 0 0 0 + + + +
---------------------
1 1 1 0 0 1 0 0 1 0
```

→ (-110)

→ 2's complement to verify

```
0 0 0 1 1 0 1 1 0 1
                  1
---------------------
0 0 0 1 1 0 1 1 1 0   = (110)
```

__/__/____

◉ For a negative multiplier, a straight forward solution is to form the 2's compliment of both the multiplier and the multiplicand, and proceed as in the case of postive multiplier.

↱ multiplicand

• (-13) x (-11) → multiplier.

↓ ↓

10011 10101

↓ ↓ ⟹ 2's complement.

01101 0101**0**1

```
        0 1 1 0 1
        0 1 0 1 1
        ---------
        0 1 1 0 1
      0 1 1 0 1 +
    0 0 0 0 0 + +
  0 1 1 0 1 + + +
0 0 0 0 0 + + + +
---------------------
0 1 0 0 0 1 1 1 1   ⟹ 143
```

0 1 0 0 0 1 1 1 1
(128, 8, 4, 2, 1)

128 + 8 + 4 + 2 + 1

= 143

⊛ Booth Algorithm

Multiplier		Multiplicand version selected by bit i
Bit i	Bit i+1	
0	0	0 * M
0	1	1 * M
1	0	-1 * M
1	1	0 * M

Example of Booth Algorithm

+1 -1

0 0 1 0 1 1 0 0 1 1 1 0 1 0

0 1 -1 +1 0 -1 0 +1 0 0 -1 +1 -1 0

0 1 0 1 1 0 1 → 0 0 1 1 1 1 0

0 0 1 1 1 1 0 → 0 +1 0 0 0 -1 0

0 1 0 1 1 0 1 +1 ⇒ Multiplicand

0 +1 0 0 0 -1 0 -1 ⇒ 2's complement of multiplicand

+1 ⇒ 0 1 0 1 1 0 1

-1 ⇒ 1 0 1 0 0 1 1

0 0 1 0 1 1 0 1 0 0 0 1 0 1 0 0 1 1 0

X

0 1 -1 1 0 -1 1 -1 0 1 -1 1 -1 0 1 0 -1 0

X

0 1 0 1 1 0 1

0 +1 0 0 0 -1 0

0 0 0 0 0 0 0

0 0 1 0 1 0 1 + +

0 0 0 0 0 0 0 + + +

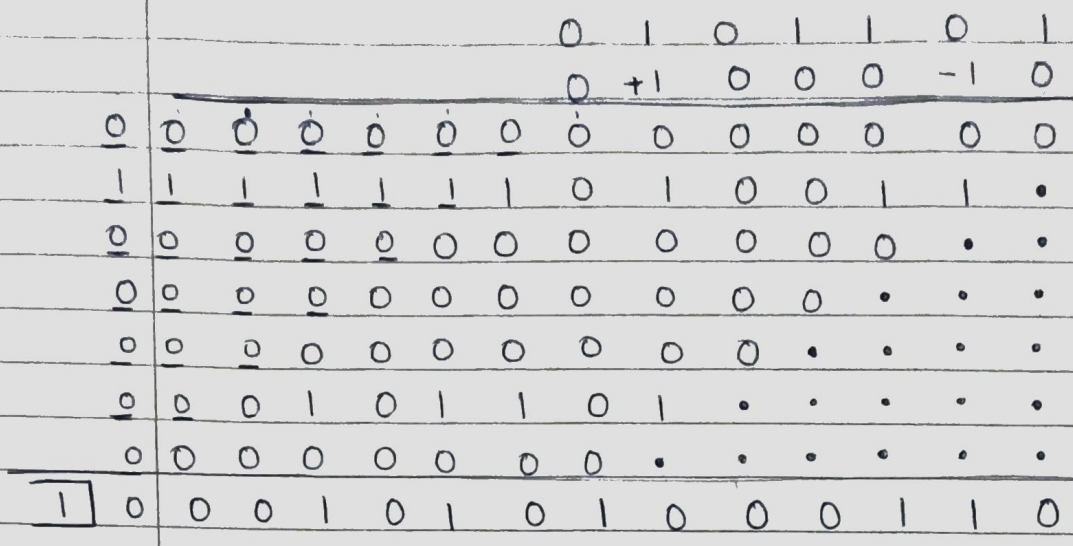
0 0 0 0 0 0 0 + + + +

0 0 0 0 0 0 0 + + + + +

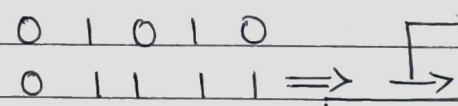
0 1 0 1 1 0 1 + + + + + +

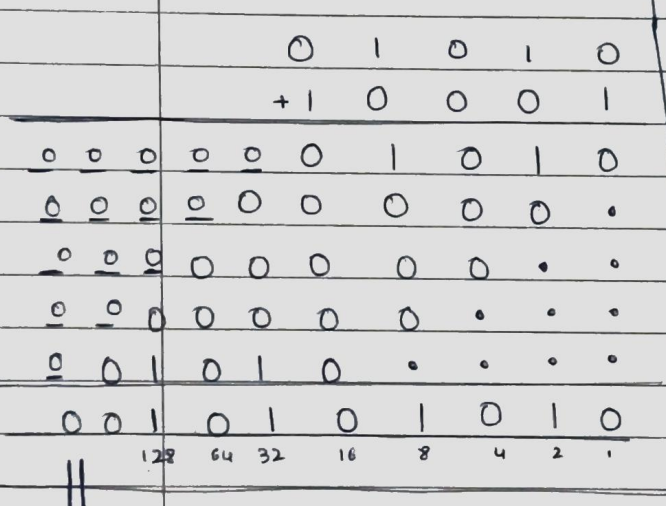
0 0 1 0 1 1 1 1 0 1 0

Next page



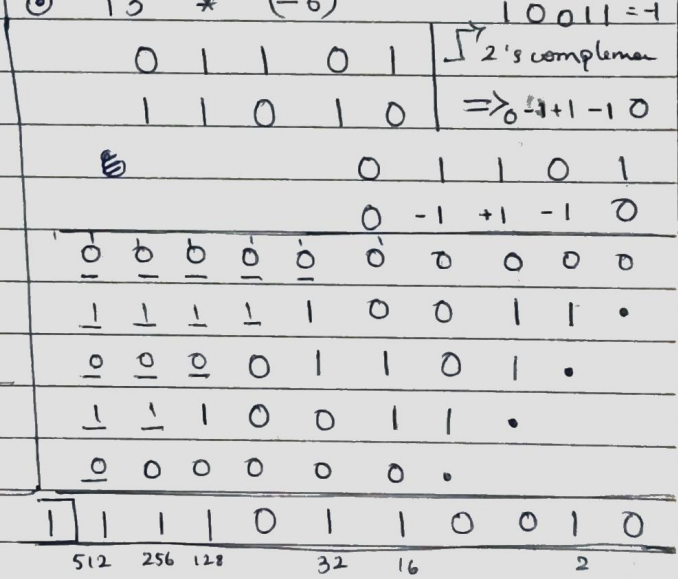
⊙ 10 × 15





2's complement

⊙ 13 * (−6)



2's complement

0 0 0 1 0 0 1 1 1 0 ⇒ 78
64 8 4 2

④ Bit-pair Recoding of Multipliers

The equation to be applied for bit-pair recoding multiplier is :- $i * 2^1 + j * 2^0 \Rightarrow i*2+j$

Steps :-

1) Apply Booth's algorithm
2) Apply bit-pair recoding equation

Ex :- 1 1 1 0 1 0 [0] => Implied 0 to right of LSB.

0 0 -1 +1 -1 0

0 -1 -2

- $-1 * 2 + 0 \Rightarrow -2$
- $-1 * 2 + 1 \Rightarrow -2 + 1 \Rightarrow -1$
- $0 + 0 \Rightarrow 0$

⑤

0 1 1 0 1 => 2's complement => 10011

0 -1 -2 (-2) => (-1) + (-1)

10011
10011
100110

1 1 1 1 0 0 1 1 0
1 1 1 0 0 1 1 ++
0 0 0 0 0 0 + + + +
1 [1] 1 0 1 1 0 0 1 0

Absent → 23/12/24

⑥ Carry Save Adder

A = 45						B = 25						C = 20					
2^5	2^4	2^3	2^2	2^1	2^0	2^5	2^4	2^3	2^2	2^1	2^0	2^5	2^4	2^3	2^2	2^1	2^0
32	16	8	4	2	1	32	16	8	4	2	1	32	16	8	4	2	1
1	0	1	1	0	1	0	1	1	0	0	1	0	1	0	1	0	0

1 0 1 1 0 1 — 20
0 1 1 0 0 1 — 25
0 1 0 1 0 0 — 45
1 0 0 0 0 0 — 90

Sum => 1 0 0 0 0 0
Carry => 0 1 1 1 0 1 0
1 0 1 1 0 1 0 => 90

__/__/__

•

	1 0 1 1 0 1	45
	1 1 1 1 1 1 1	63
	1 0 1 1 0 1	A
	1 0 1 1 0 1 .	B
	1 0 1 1 0 1 . .	C
	1 0 1 1 0 1 . . .	D
	1 0 1 1 0 1	E
	1 0 1 1 0 1	F
1 0 0 1	0 0 1 0 0 0 1 1	X
1 0 1 1	0 0 0 1 0 0 1 1	⟹ 2835

X

A	1 0 1 1 0 1
B	1 0 1 1 0 1
C	1 0 1 1 0 1
S_1	0 1 1 0 1
C_1	1 1 0 1 0

X

Carry Save method

A	1 0 1 1 0 1	D	1 0 1 1 0 1
B	1 0 1 1 0 1 .	E	1 0 1 1 0 1 .
C	1 0 1 1 0 1 . .	F	1 0 1 1 0 1 . .
S_1	1 1 0 0 0 0 1 1	S_2	1 1 0 0 0 0 1 1
C_1	0 0 1 1 1 1 0 0 0	C_2	0 0 1 1 1 1 0 0 0

S_1	1 1 0 0 0 0 1 1	S_3	1 1 0 0 0 1 0 0 0 1 1
C_1	0 0 1 1 1 1 0 0 .	C_3	0 0 0 0 1 0 1 1 0 0 0 .
S_2	1 1 0 0 0 0 1 1 . . .	C_2	0 0 1 1 1 1 0 0 . . . .
S_3	1 1 0 1 0 1 0 0 0 1 1	S_4	0 1 0 1 1 1 1 0 1 0 0 1 1
C_3	0 0 0 0 1 0 1 0 0 0	C_4	0 0 1 0 1 0 1 0 0 0 0 0 .

Result ⟸ 0 1 0 1 1 0 0 0 1 0 0 1 1

•

	0 1 0 1 0 0	20 (10100)
	0 1 1 0 0 1	25 (11001)
	0 1 0 1 0 0	A
	0 0 0 0 0 0 .	B
	0 0 0 0 0 0 . .	C
0	1 0 1 0 0 .	D
0	1 0 1 0 0 .	E
0	0 0 0 0 .	F

A	0 1 0 1 0 0
B	0 0 0 0 0 0
C	0 0 0 0 0 0 . .
Sum_1	0 0 0 1 0 1 0 0
C_1	0 0 0 0 0 0 0 0 0

D	0 1 0 1 0 0	0 0 0 1 0 1 0 0
E	0 1 0 1 0 0	0 0 0 0 0 0 0 0
F	0 0 0 0 0 0 0	0 0 1 1 1 1 0 0
S_2	0 0 1 1 1 1 0 0	S_3 0 0 1 1 1 1 0 1 0 0
C_2	0 0 0 0 0 0 0 0	C_3 0 0 0 0 0 0 0 0 0 0

S_3	0 0 1 1 1 1 1 0 1 0 0
C_3	0 0 0 0 0 0 0 0 0 0 0
C_2	0 0 0 0 0 0 0 0 0 0 0
S_4	0 0 0 1 1 1 1 1 0 1 0 0
C_4	0 0 0 0 0 0 0 0 0 0 0 0
	0 0 0 1 1 1 1 0 1 0 0

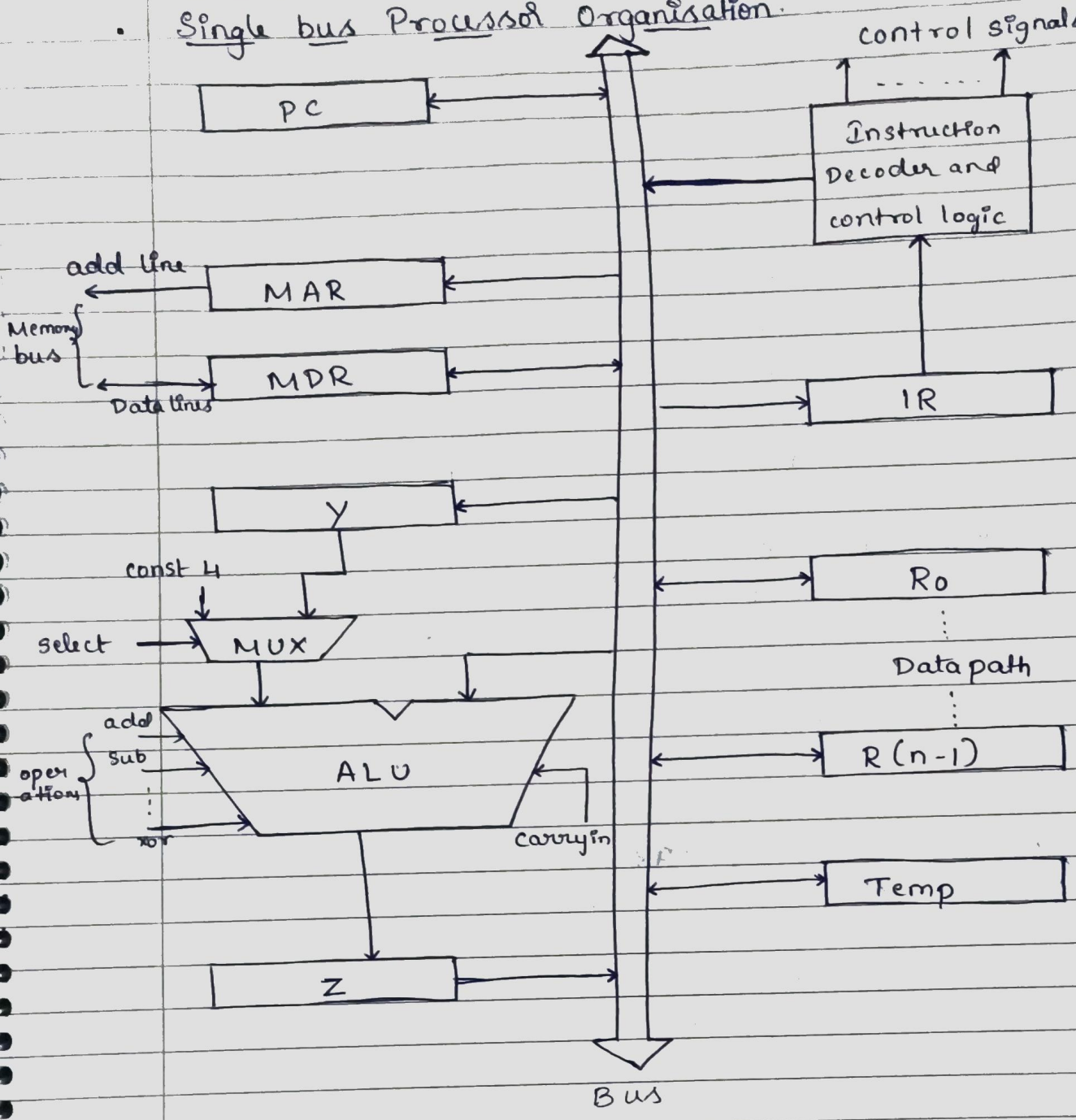
=> Result
=> 500

Theory part of unit - 4:

- Fundamental Concepts
  - → Processor fetches one instruction at a time and perform the operation specified.
  - → Processor keeps track of the address of the memory location containing the next instruction to be executed using PC (Program Counter).
  - → IR (Instruction Register).

- Executing an Instruction
  - → IR ← [[PC]]
  - → PC ← [PC] + 4
  - → Carry out the actions specified by the instruction in IR (execution phase).

- Single bus Processor Organisation.



(Passing Instruction, signals from one-unit to another)

With few exception, an instruction can be executed by performing one or more of the following operations in some specified sequence.

- Transfer a word of data from one processor register to another or to the ALU.
- Perform an arithmetic or a logic operation and store the result in a processor register.
- Fetch the contents of a given memory location and then load them into a processor register.
- Store a word of data from a processor register into a given memory location.

* Adding content of two register [R_1 & R_2] and storing it in another register [R_3].
  - R_1 out , Yin
  - R_2 out , select Y , Add, Zin
  - Z out , R_3 in

3. * Move (R_1) , R_2
  - R_1 out , MARin , Read
  - MDRinE , WMFC
  - MDRout , R_2in.

4. * Move R_2, (R_1)
  - R_1 out , MARin , Read
  - R_2 out , MDRin , WMFC
  - MDR out , ~~R_1 in~~ , WMFC.

* Add (R_3), R_1
  - R_3 out , MARin

continues. next page →

__/__/__

④ Add (R3), R1

- PCout, MARin, Read, Select 4, Add, Zin.
- Zout, PCin, Y. WMFC.
- MDRout, IRin
- R3out, MARin, Read
- R1out, Yin, WMFC
- MDRout, Select Y, Add, Zin
- Zout, R1in, End.

⑤ Execution of Branch Instruction

- PCout, MARin, Read, select 4, Add, Zin.
- Zout, PCin, Yin, WMFC.
- MDRout, IRin.
- Offset-field-of-IRout, Add, Zin
- Zout, PCin, End.

Multiple Bus Organisation

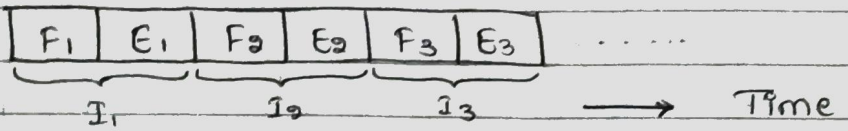
- Allow the content of 2 different registers to ~~do~~ be accessed simultaneously & have their content placed on buses A and B.
- Allow data on bus C to be loaded into 3rd register during the same clock cycle.
- Increment unit/circuit is placed
- ALU simply passes one of its i/p operands unmodified to bus.

→ Control signal : R = A or R = B.

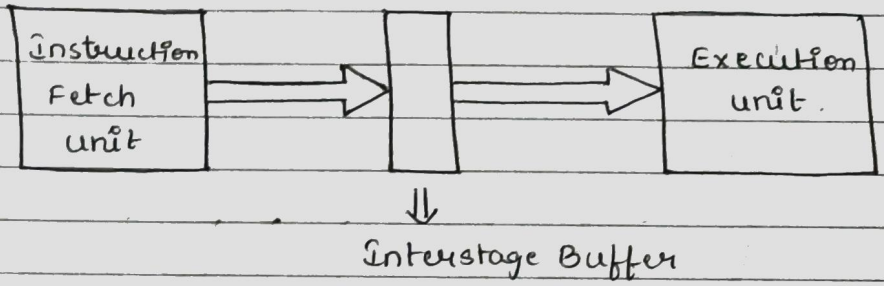
__/__/____

④ Pipelining.

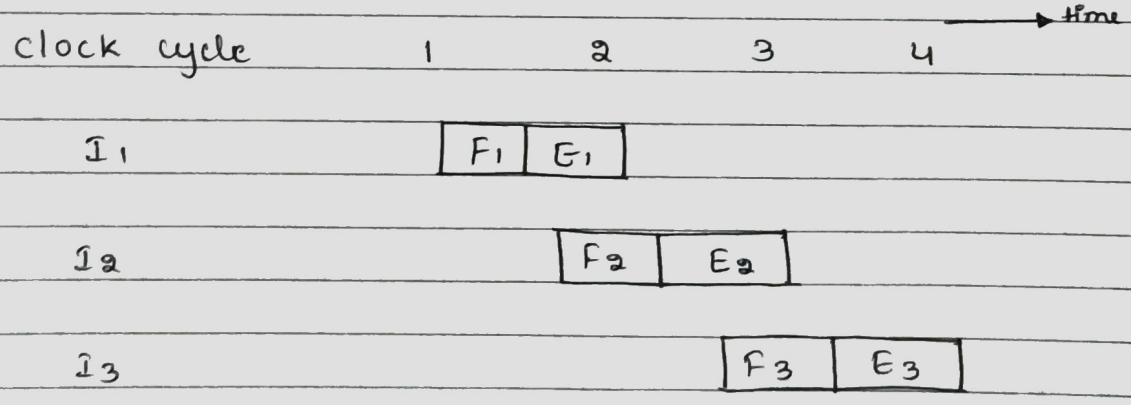
Sequential equation



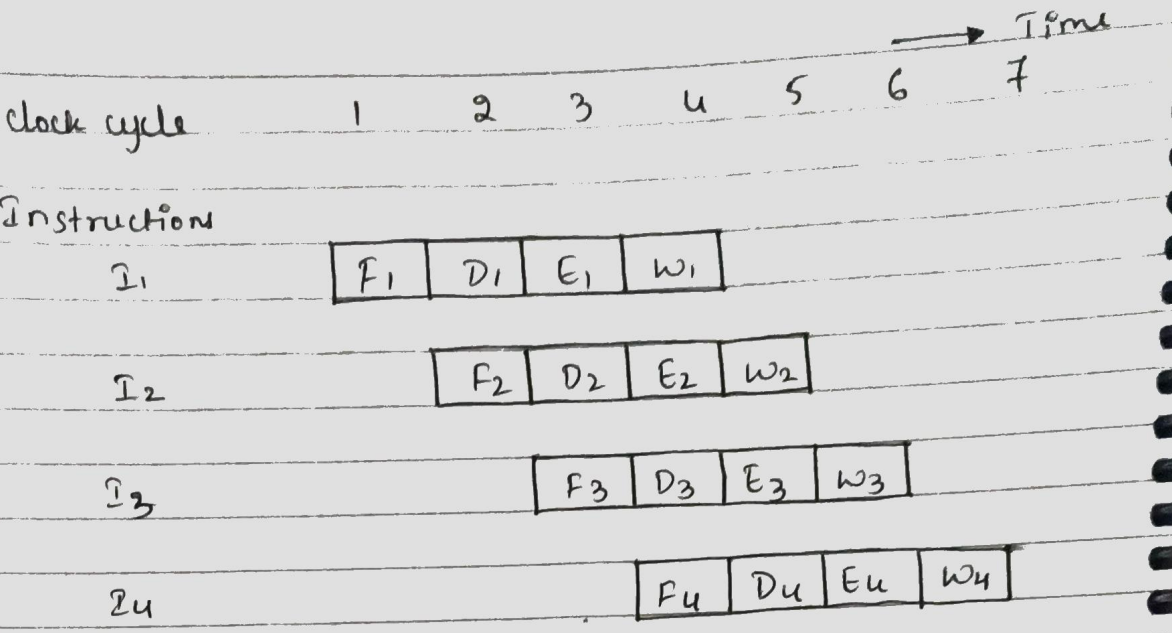
Hardware organization



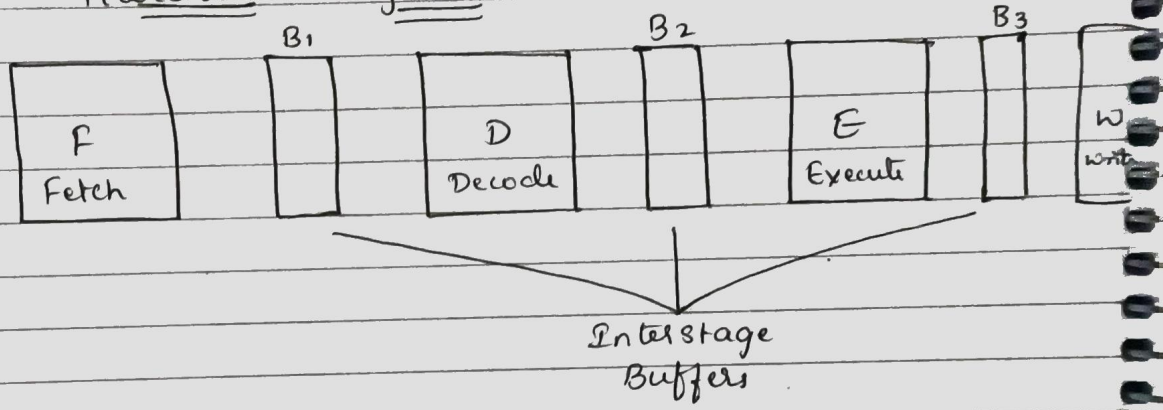
Pipelined Execution



- The process of an instruction in 4 steps
  F - Fetch [read the istruction from the memory]
  D - Decode [decode instruction & fetch source operand]
  E - Execute [performs operation specified by the instruc]
  W - Write [stores the result in destination location].



Hardware organisation



Condition that causes pipeline to stall is called hazard.

⊛ 3 types of hazards in pipelining performance :-

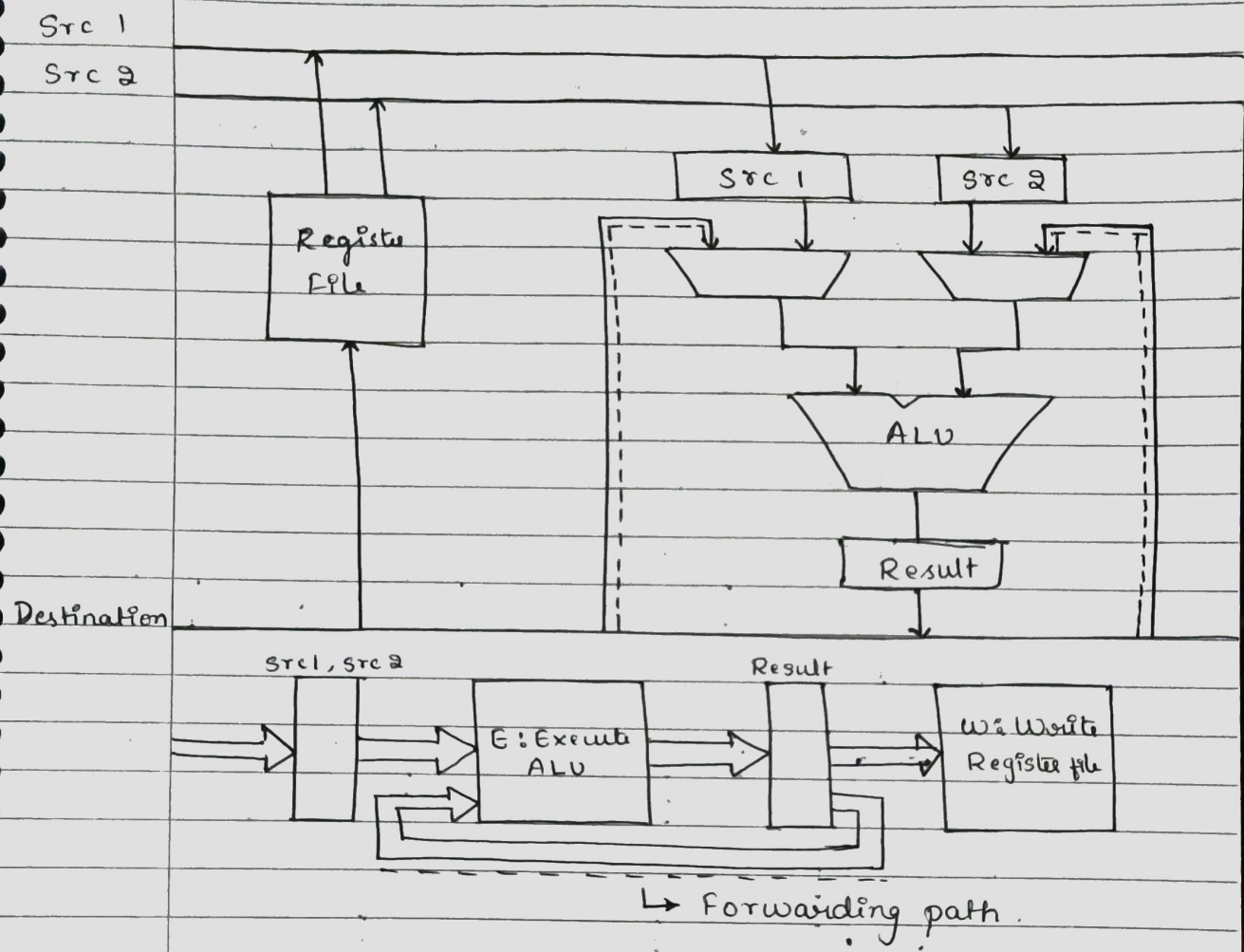
- Data hazard - data is not available.
- Control hazard - delay in availability of instruction.
- Structure hazard - when two instruction require hardware at the same time (access of memory).

Instruction hazard

Data dependencies :- the data / input of one instruction is dependent on another instruction.

__/__/____

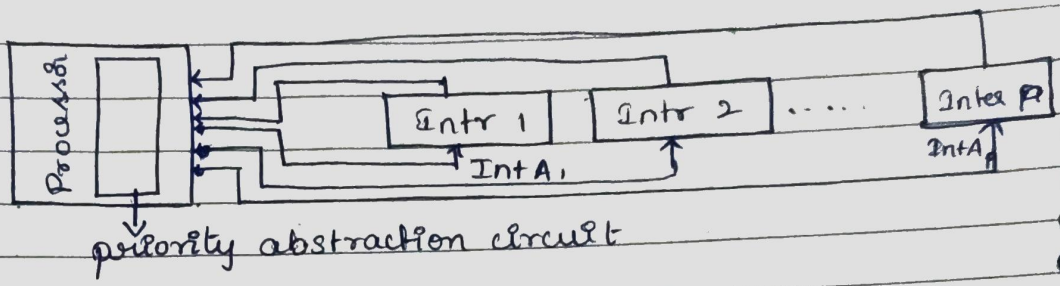
Operand forwarding



Absent cont

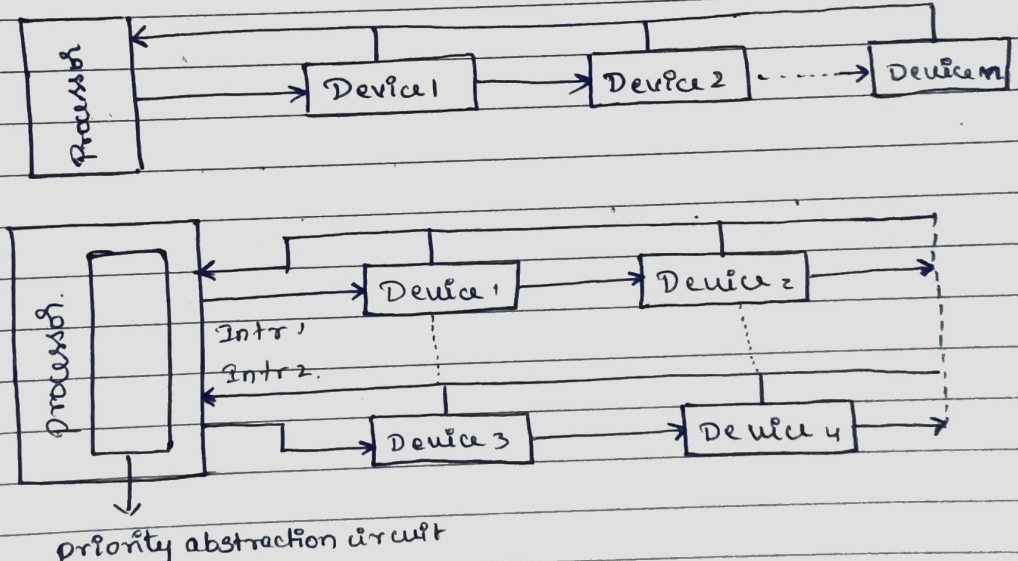
Unit - 5

Interrupt Nesting



If processor is handling one interrupt, it should respond to the next interrupt based on priority

Simultaneous Request.



Controlling device request

- Interrupt request by only those devices which are used by the programs.
- At the device end, an interrupt enable bit in a control register determines whether the device is allowed to ~~cotts~~ generate an interrupt request.

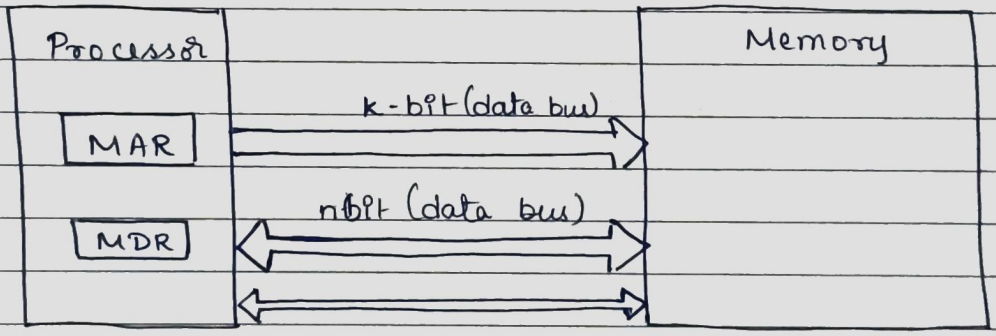
__/__/____

- At the process end, either an interrupt-enable bit in the PS register or a priority structure determines whether a given interrupt request will be accepted.

Memory System

- The max size of the memory that can be used in any computer is determined by the addressing scheme.
  16-bit addressing = 2^{16} = memory locations
- Most modern computers are byte addressable.
  * Big endian * Little endian.

Traditional Architecture



- "Block transfer" - bulk data transfer - starting address
- Memory access time - The time elapsed b/w the initiation of a memory Read or Write operation and the completion of that operation.
- Memory cycle time - Minimum time delay require b/w the initiation of two successive memory operation.

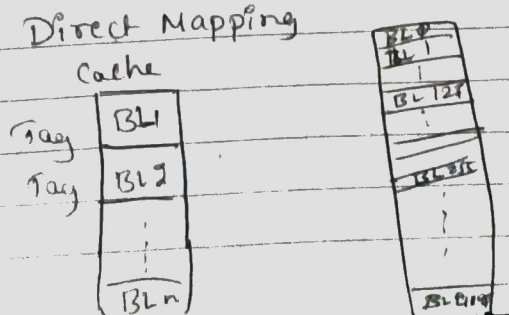
- RAM
- Cache memory - temperory memory present in processor which gives fastest access to the memory data.
- Virtual memory -
- Memory Management unit (MMU) - A special hardware unit that translates virtual memory into physical memory.

Cache Memory

- Cache block - cache line.
  A set of contiguous address locations of some size.
- Locality of reference
  - temporal -
  - spatial - instruction
- Speed of main memory < speed of cache memory.
- Replacement algorithm [when cache is full].
  → Dirty bit / modified bit - overwriting the content of cache [write through / back]

⊛ Address Mapping
- Direct Mapping
- Associative Mapping
- Set Associative Mapping.

⊛ Direct Mapping



Tag	Block word
5	7	4

$2^{12} = 4096$

$2^7 = 128$ [cache mem]

$4096 / 128 = \underline{32}$ Tags.

__/__/__

Ex: 000 00500 Address